

《数据库系统原理》实验报告（实验六）					
题目：实验六：miniOB 进阶实验					
学号	2351520	姓名	毛星博	日期	2025.5.18
实验环境：Docker OceanBase-miniob					

实验一、select-table

1.赛题基本信息

当前系统支持单表查询的功能，需要在此基础上支持多张表的笛卡尔积关联查询。需要实现 `select * from t1,t2; select t1.,t2. from t1,t2;`以及 `select t1.id,t2.id from t1,t2;`查询可能会带条件。查询结果展示格式参考单表查询。注意查询条件中的“不等”比较，除了"`<>`"还要考虑"`!=`" 比较符号。每一列必须带有表信息，比如:

t1.id | t2.id ; 1 | 1

2.实现思路分析

在 Binder 阶段给每个字段名加上表前缀（`table_name.field_name`），保证输出列名带表信息。

在 ExpressionBinder 把 FieldExpr 生成时，就把表名和列名拼成 `t1.id`、`t2.name`，这样后续任何输出都会带前缀。

在逻辑计划生成器里已有多表笛卡尔积（交叉 join）的框架，只需在物理计划生成器里把子算子都挂到 NestedLoopJoinPhysicalOperator 上。

在 SQL 词法/语法文件中把 `!=` 也映射到同一个 `CompOp::NOT_EQUAL`，上层表达式评估时会和 `<>` 一样处理。

最终输出由 Project 阶段根据 `Expression::name()` 打表头，数据按嵌套循环算子读到的行顺序展示。

1.修改 expression_binder.cpp 中 bind_unbound_field_expression:

```
RC ExpressionBinder::bind_unbound_field_expression(
    unique_ptr<Expression> &expr, vector<unique_ptr<Expression>> &bound_expressions)
{
    // ...existing code...

    const FieldMeta *field_meta = table->table_meta().field(field_name);
    if (nullptr == field_meta) {
        LOG_INFO("no such field in table: %s.%s", table_name, field_name);
        return RC::SCHEMA_FIELD_MISSING;
    }

    Field field(table, field_meta);
    FieldExpr *field_expr = new FieldExpr(field);
    // 设置带表前缀的列名
    string fullname = string(table_name) + "." + field_name;
    field_expr->set_name(fullname.c_str());
    bound_expressions.emplace_back(field_expr);

    return RC::SUCCESS;
}
```

2.同样在 wildcard_fields 里给所有展开的列加表前缀:

```
static void wildcard_fields(Table *table, vector<unique_ptr<Expression>> &expressions)
{
    const char *table_name = /* ... */;
    // 遍历表的所有列元信息
    for (int i = 0; i < table->table_meta().field_count(); i++) {
        const FieldMeta *fmeta = table->table_meta().field(i);
        Field field(table, fmeta);
        FieldExpr *fe = new FieldExpr(field);
        // 加上前缀 t.col
        string fullname = string(table_name) + "." + fmeta->name().c_str();
        fe->set_name(fullname.c_str());
        expressions.emplace_back(fe);
    }
}
```

3.修改 physical_plan_generator.cpp 中两表交叉 join 的物理算子挂载:

```
RC PhysicalPlanGenerator::create_plan(JoinLogicalOperator &join_oper,
                                     unique_ptr<PhysicalOperator> &oper,
                                     Session* session)
{
    // ...existing code...
    // 走嵌套循环
    unique_ptr<PhysicalOperator> join_physical_oper(new NestedLoopJoinPhysicalOperator());
    for (auto &child_oper : join_oper.children()) {
        unique_ptr<PhysicalOperator> child_physical_oper;
        RC rc = create(*child_oper, child_physical_oper, session);
        if (rc != RC::SUCCESS) {
            LOG_WARN("failed to create child oper for join. rc=%s", strrc(rc));
            return rc;
        }
        // 把每个子算子都添加到嵌套循环算子上, 得到笛卡尔积
        join_physical_oper->add_child(std::move(child_physical_oper));
    }
    oper = std::move(join_physical_oper);
    return RC::SUCCESS;
}
```

3.实验结果

2025-05-18 22:01	https://gitee.com/moxianbao88...	main	855d208	● 执行成功
select-tables	10	10	-	

实验二、drop-table

1.赛题基本信息

删除表。清除表相关的资源。

注意：要删除所有与表关联的数据，不仅仅是在 create table 时创建的资源，还包括索引等数据。

2.实现思路分析

drop table 与 create table 相反，要清理掉所有创建表和表相关联的资源，比如描述表的文件、数据文件以及索引等相关数据和文件。

sql 流转到 default_storge 阶段的时候，在处理 sql 的函数中，新增一个 drop_table 的 case。

drop table 就是删除表，在 create table 时，会新建一个 t.table 文件，同时为了存储数据也会新建一个 t.data 文件存储下来。同时创建索引的时候，也会创建记录索引数据的文件，在删除表时也要一起删除掉。

那么删除表，就需要删除 t.table 文件、t.data 文件和关联的索引文件。

同时由于 buffer pool 的存在，在新建表和插入数据的时候，会写入 buffer pool 缓存。所以 drop table，不仅需要删除文件，也需要清空 buffer pool，防止在数据没落盘的时候，再建立同名表，仍然可以查询到数据。

如果建立了索引，比如 t_id on t(id)，那么也会新建一个 t_id.index 文件，也需要删除这个文件。

这些东西全部清空，那么就完成了 drop table。

1.在 default_storage_stage.cpp 中的处理 SQL 语句的 case 中增加一个

```
case SCF_DROP_TABLE: {
    const DropTable& drop_table = sql->sstr[sql->q_size-1].drop_table; // 拿到要 drop 的表
    rc = handler_->drop_table(current_db,drop_table.relation_name); // 调用 drop table 接口.drop table 要在 handler
    中实现
    snprintf(response,sizeof(response),"%s\n", rc == RC::SUCCESS ? "SUCCESS" : "FAILURE"); // 返回结果，带不带
    换行符都可以
}
break;
```

2.在 default_handler.cpp 文件中，实现 handler 的 drop_table 接口：

```
RC DefaultHandler::drop_table(const char *dbname, const char *relation_name) {
    Db *db = find_db(dbname); // 这是原有的代码，用来查找对应的数据库，不过目前只有一个库
    if(db == nullptr) {
        return RC::SCHEMA_DB_NOT_OPENED;
    }
    return db->drop_table(relation_name); // 直接调用 db 的删掉接口
}
```

3.在 db.cpp 中，实现 drop_table 接口

```
RC Db::drop_table(const char* table_name)
{
    auto it = opened_tables_.find(table_name);
    if (it == opened_tables_.end())
    {
        return SCHEMA_TABLE_NOT_EXIST; // 找不到表，要返回错误，测试程序中也会校验这种场景
    }
    Table* table = it->second;
    RC rc = table->destroy(path_.c_str()); // 让表自己销毁资源
    if(rc != RC::SUCCESS) return rc;
}
```

```
opened_tables_.erase(it); // 删除成功的话，从表 list 中将它删除
delete table;
return RC::SUCCESS;
}
```

4.table.cpp 中清理文件和相关数据

```
RC Table::destroy(const char* dir) {
    RC rc = sync(); //刷新所有脏页

    if(rc != RC::SUCCESS) return rc;
    std::string path = table_meta_file(dir, name());
    if(unlink(path.c_str()) != 0) {
        LOG_ERROR("Failed to remove meta file=%s, errno=%d", path.c_str(), errno);
        return RC::GENERIC_ERROR;
    }

    std::string data_file = std::string(dir) + "/" + name() + TABLE_DATA_SUFFIX;
    if(unlink(data_file.c_str()) != 0) { // 删除描述表元数据的文件
        LOG_ERROR("Failed to remove data file=%s, errno=%d", data_file.c_str(), errno);
        return RC::GENERIC_ERROR;
    }

    std::string text_data_file = std::string(dir) + "/" + name() + TABLE_TEXT_DATA_SUFFIX;
    if(unlink(text_data_file.c_str()) != 0) { // 删除表实现 text 字段的数据文件（后续实现了 text case 时需要考虑，
    最开始可以不考虑这个逻辑）
        LOG_ERROR("Failed to remove text data file=%s, errno=%d", text_data_file.c_str(), errno);
        return RC::GENERIC_ERROR;
    }
}
```

3.实验结果

2025-05-18 22:01	https://gitee.com/moxianbao88...	main	855d208	● 执行成功
drop-table		10	10	-

出现的问题：无

解决方案：无